

Report

CAP 931: Capstone - Build a Sales Agent Prototype Using Multi-Agent GPT Models

Eduard Lukyanov
August 17th, 2026

Overview	2
Steps	3
Conclusion	11
Appendix	12

Overview

This capstone developed a Sales Account Intelligence Agent, a Streamlit-based prototype that helps sales representatives turn public prospect website information into a structured account intelligence brief. The application collects product information, a prospect company website, a target buyer, optional competitor URLs, and an optional product overview document.

The app retrieves public webpage content, extracts readable text, optionally extracts text from product documents, and prepares the information for account research and outreach generation. The primary LLM path uses OpenAI `gpt-4o-mini` to generate a concise Markdown account brief.

During testing, the OpenAI account returned an insufficient-quota error because API billing credits were unavailable. To keep the prototype usable and transparent, a clearly labeled rule-based fallback was added. The fallback is not presented as AI-generated. It formats retrieved public webpage text and user inputs into an account-brief structure.

Project Goals

- Provide a Streamlit interface for entering product, prospect, competitor, and buyer information.
- Retrieve and parse public webpage content from a prospect URL.
- Process optional competitor URLs and an optional product-overview document.
- Use structured prompts and an LLM to generate a research-based account brief.
- Include strategy, competitor, leadership, product-fit, outreach, and source-link sections.
- Experiment with more than one prompt structure.
- Handle API and webpage-retrieval failures gracefully.
- Document future production improvements, including an alert system and deployment considerations.

Steps

1. Project Setup and Dependency Management

The project was implemented in Python and managed with `uv` rather than raw `pip`. The `pyproject.toml` file stores project dependencies, while `uv.lock` stores resolved versions for reproducible installation.

The project uses the following dependencies:

- **Streamlit** for the web interface.
- **requests** for public HTTP webpage retrieval.
- **BeautifulSoup** from `beautifulsoup4` for HTML parsing and readable-text extraction.
- **PyPDF2** for extracting text from uploaded PDF product-overview files.
- **OpenAI** for the LLM client and account-brief generation.
- **python-dotenv** for loading the `OPENAI_API_KEY` environment variable from a local `.env` file.
- **uv** for dependency management, virtual environment setup, and reproducible execution.

The app is started with:

```
uv run streamlit run app.py
```

The local `.env` file is excluded from Git so API credentials are not committed to the repository.

2. Streamlit Interface

The Streamlit form captures the sales context needed to generate a targeted account intelligence brief. Required inputs are validated before webpage retrieval begins.

The interface includes:

- Product name
- Product category

- Value proposition
- Prospect company website
- Target buyer
- Competitor URLs, optional, one URL per line
- Product overview sheet or deck upload, optional, supporting `.txt`, `.md`, and `.pdf`
- Prompt version selector: V2 Assignment Aligned or V1 Simple Brief

The application normalizes URLs by adding `https://` when needed. It validates that the prospect URL contains both a valid scheme and network location before making a public HTTP request.

The current prototype uses required structured form fields instead of conversational follow-up questions. When essential information is missing, the app prompts the user to complete the required field before research begins.

3. Public-Web Data Integration

The application retrieves the prospect webpage with `requests` and a browser-like User-Agent header. BeautifulSoup parses returned HTML and removes scripts, styles, navigation, headers, footers, and sidebars before extracting readable text.

The application stores the webpage title and limits prospect-page text to 5,000 characters for prompt context. This keeps the LLM request focused on relevant readable content rather than raw HTML.

When competitor URLs are supplied, the app retrieves each competitor webpage using the same extraction process. The retrieved competitor text is included in the V2 research context.

For the optional product-document upload:

- `.txt` and `.md` files are decoded and read as text.
- `.pdf` files are processed with PyPDF2.
- The prototype extracts text from up to the first 10 PDF pages.
- Uploaded text is limited before it is passed into the prompt.

This workflow allows a sales representative to provide product-specific context without embedding product details directly in the code.

4. LLM Capabilities

Model Selection

The primary LLM path uses OpenAI `gpt-4o-mini`. It was selected because it provides a practical balance of response quality, speed, and cost for a prototype that produces concise and structured sales briefs.

The model is intended to summarize supplied prospect and competitor context, maintain a defined Markdown structure, identify relevant strategy and leadership signals when supported by the sources, and draft concise outreach language.

A limitation discovered during testing was API billing availability. The application reached the OpenAI request stage, but the API returned an insufficient-quota error because the account had no available API credits. This was an account-billing limitation rather than a failure of the Streamlit, webpage-retrieval, or prompt-building code.

Prompt Engineering and Experimentation

The final prototype includes two prompt versions to support prompt experimentation.

V1 — Simple Brief is the baseline prompt. It uses product information and prospect webpage content to generate a short brief with:

- Prospect Snapshot
- Likely Business Focus
- Sales Angle
- Suggested Outreach Message
- Source Used

V2 — Assignment Aligned is the default prompt. It includes:

- Product name, category, and value proposition
- Prospect website title and extracted text

- Target buyer
- Optional competitor webpage text
- Optional uploaded product notes
- Source URLs used for research

V2 requests a fixed Markdown output with:

- Company Strategy
- Competitor Mentions
- Leadership Information
- Product/Strategy Summary
- Recommended Sales Angle
- Suggested Outreach Message
- Article Links

Both prompts define the model's role as a careful B2B sales research assistant. Both instruct the model to use only supplied source material and to state **“Not found in supplied source.”** when information is not supported. The outreach email is limited to fewer than 120 words.

The V1-versus-V2 design is a prompt experiment. V1 provides a simple baseline. V2 improves usefulness by adding competitor context, uploaded product documentation, leadership and strategy categories, sales-angle bullets, and source links.

Because API credits were exhausted, a full live-output comparison between V1 and V2 could not be completed during testing. However, both prompt paths, their structured requirements, and the fallback behavior were implemented.

Data Integration

The prospect URL is used to retrieve public webpage text. Competitor URLs are optionally retrieved and added to the V2 prompt. Uploaded product notes can also supplement the LLM context.

This approach grounds the intended LLM output in supplied public content and product information rather than asking the model to produce a generic sales brief without source context.

5. Error Handling and Fallback

The app verifies that `OPENAI_API_KEY` is available before allowing LLM generation. It handles missing required fields, invalid URLs, webpage-retrieval errors, uploaded-file parsing errors, and OpenAI request errors.

When an OpenAI response contains `insufficient_quota` or `credit_balance_exhausted`, the application displays a clear user-facing warning rather than a raw technical error.

A rule-based fallback account brief is generated only when live LLM generation cannot complete because of an API quota or credit issue. The fallback uses:

- Retrieved webpage title and text
- Product name and category
- Value proposition
- Target buyer
- Competitor URLs
- Uploaded product-note text when available
- Research source URLs

The fallback is explicitly labeled as generated without live AI. This preserves transparency and distinguishes it from a GPT-generated output.

6. Time Management

The capstone was completed in a four-day project window. Time was allocated first to establishing a working Python and Streamlit environment, then to building the form and public-web retrieval workflow.

Next, time was spent building structured prompts, OpenAI integration, and error handling. When the API quota issue was discovered, time was allocated to build a transparent fallback output so the end-to-end workflow could still be demonstrated.

The remaining time was used to add assignment-aligned enhancements: competitor URL retrieval, optional document parsing, V1/V2 prompt options, source-link display, Git version control, testing, and documentation.

This sequence prioritized a functioning end-to-end prototype before adding enhancements. It reduced project risk because the app could still demonstrate validated inputs, webpage retrieval, text processing, and structured output when the external LLM service was unavailable.

7. Testing Evidence

Testing confirmed the following workflow:

1. The Streamlit application launched successfully at `localhost:8501` using `uv run streamlit run app.py`.
2. The form accepted required product and prospect inputs.
3. The application validated required fields and prospect URLs before making requests.
4. A Salesforce prospect webpage was retrieved successfully.
5. The app displayed the prospect source page title and URL.
6. The OpenAI request path was reached but returned a 429 insufficient-quota error because the API account had no available credits.
7. The user-facing quota message displayed clearly.
8. The fallback account brief generated successfully after the quota condition was handled.

9. The fallback output remained clearly labeled as fallback output rather than live AI output.

Additional testing is planned for a second prospect and product context, missing required fields, invalid URLs, and unreachable URLs. This validates that the app behaves predictably when inputs change rather than only working for one ideal example.

Optional Enhancements

Improving Output Results

The final prototype improves output relevance by requiring source-grounded responses. The prompt tells the model not to invent facts and to use **“Not found in supplied source.”** when evidence is not present.

Future accuracy improvements could include:

- Per-finding citations tied to source URLs and extracted text snippets.
- Independent web search for recent news, press releases, job postings, annual reports, and investor filings.
- Structured side-by-side competitor comparison.
- Retrieval ranking so the most relevant source passages are sent to the model.
- Human review before using AI-generated claims in sales outreach.

Alert System Design

A future alert system could monitor selected prospect sources for user-defined keywords, such as “data platform,” “cloud migration,” “sales operations,” or a competitor’s name.

A possible workflow would be:

1. The user saves a prospect company, source URLs, and monitored keywords.
2. A scheduled job checks press-release pages, career pages, company-news pages, and relevant public sources.
3. The system compares newly retrieved content with previously stored results.
4. When new content contains monitored keywords, the system sends an email alert.

5. The alert includes a link, source title, short snippet, date found, and suggested relevance to the sales opportunity.

Production Deployment Considerations

A production version could be deployed with Streamlit Community Cloud, a containerized cloud platform, or a managed cloud service.

Production planning would include:

- Store API keys in a secrets manager rather than local `.env` files.
- Add authentication and role-based access controls.
- Add rate limiting to prevent abuse and unexpected API costs.
- Add request logging, monitoring, and error tracking.
- Add caching for retrieved webpages to reduce duplicate requests and cost.
- Add retry logic for temporary network and API failures.
- Respect website terms of service, robots rules, privacy obligations, and data-handling standards.
- Add automated tests for URL validation, file parsing, webpage retrieval, prompts, and fallback behavior.
- Use secure deployment configuration and regular dependency updates.

Additional Model Workflows

Different model workflows could support sales representatives beyond the account brief.

For example:

- A language model can produce a meeting brief from approved account research and user-provided notes.
- A document-processing workflow can extract key value points from product sheets, pitch decks, and customer case studies.

- A presentation-generation workflow can create a draft sales-deck outline from approved research, product materials, and account priorities.
- A retrieval-based workflow can identify supporting evidence from source documents before a model drafts talking points.
- A human reviewer can approve or revise claims before content is sent to a prospect.

Conclusion

The Sales Account Intelligence Agent demonstrates an end-to-end prototype for collecting sales inputs, retrieving public prospect-webpage data, processing competitor and optional product-document context, preparing structured LLM prompts, and presenting a one-page account intelligence output.

The project demonstrates practical engineering decisions including dependency management with `uv`, environment-variable handling, input validation, public-web parsing, optional file processing, prompt experimentation, exception handling, and transparent degradation when an external API is unavailable.

The major limitation during testing was exhausted OpenAI API billing quota. The application successfully reached the API request stage, but live LLM output could not be evaluated fully without API credits. The rule-based fallback ensured the workflow remained demonstrable, but it is accurately described as contingency output rather than LLM-generated output.

Sales Account Intelligence Agent

Enter product, prospect, competitor, and optional product-document details to create a research-based sales account brief.

Product Information

Product Name *
InsightFlow AI

Product Category *
AI-powered sales and customer intelligence platform

Value Proposition *
InsightFlow AI turns public company signals into concise, source-backed account briefs so sales teams can personalize outreach faster.

Upload product overview sheet or deck (optional)

python...e-guide.md

Prospect Information

Prospect Company Website *
salesforce.com

Target Buyer *
VP

Prospect Information

Prospect Company Website *

salesforce.com

Target Buyer *

VP

Competitor URLs (optional)

Enter one URL per line

Prompt Experimentation

Prompt Version

V2 - Assignment Aligned

Generate Account Brief

Webpage retrieved successfully.

Research Sources

Prospect page title: Salesforce: The #1 Agentic AI CRM

Prospect URL: <https://salesforce.com>

Uploaded file: python-uv-complete-guide.md

OpenAI billing quota is exhausted. Showing a fallback brief generated from supplied inputs and retrieved public webpage content.

Generated Account Brief »

Account Intelligence Brief

Company Strategy

Based on the retrieved public webpage text from **Salesforce: The #1 Agentic AI CRM**, the company appears to focus on: Salesforce: The #1 Agentic AI CRM Skip to content Start for free Talk to an expert Watch all demos Watch full demo Watch full demo Watch full demo Watch full demo Watch full demo 34 % increase in employee productivity Source: 2026 Salesforce Success Metrics Global Highlights Learn more 30 % reduction in call handling time Source: Volkswagen customer story Learn more 33 % increase in employee productivity Source: 2025 Salesforce Success Metrics Global Highlights Learn more 16 x faster delivery of agents Source: Valoir independent analyst report Learn more 39 % faster delivery of actionable analytics Source: 2025 Salesforce Success Metrics Global Highlights Learn more 25 hrs saved per rep, every week Source: 2025 Salesforce Success Metrics Global Highlights Learn more State of the Agentic Enterprise Report Get the guide Explore Data 360 Explore Customer 360 Explore Agentforce Explore Table

Competitor Mentions

Supplied competitor URLs: Not found in supplied source.

Direct competitor mentions in the supplied source: Not found in supplied source.

Leadership Information

Not found in supplied source.

Product/Strategy Summary

InsightFlow AI is a **AI-powered sales and customer intelligence platform** offering. Value proposition: InsightFlow AI turns public company signals into concise, source-backed account briefs so sales teams can personalize outreach faster.

This may be relevant to a **VP** if the prospect is focused on the business areas described in the public webpage content.

Uploaded product notes summary:

Per Scholas

The Complete Python UV Guide - From Setup to Production Version 1.2

Created by: Alexandros Karales (He/Him) Role: Technical Instructor (Lead AI & Data Analytics) Email: akarales@perscholas.org

A comprehensive guide to managing Python projects with `uv`, the ultra-fast package manager built in Rust by Astral.

Appendix

A. Commands Used

```
uv add streamlit requests beautifulsoup4
uv add openai python-dotenv pypdf2
uv sync
uv run streamlit run app.py
git status
git add app.py pyproject.toml uv.lock README.md .gitignore
```

```
git commit -m "Resolve merge conflicts and finalize  
capstone app"  
git push origin main
```

B. Evidence to Attach

- Screenshot of the Streamlit form with required fields and optional upload field
- Screenshot showing the V1/V2 prompt-version selector
- Screenshot showing successful webpage retrieval and research-source information
- Screenshot showing competitor URLs entered or processed
- Screenshot of the clean OpenAI billing-quota warning
- Screenshot of the fallback-generated account intelligence brief
- Screenshot of the Markdown download button
- Screenshot of `uv` dependency setup or `uv tree`
- Screenshot of `app.py`, `pyproject.toml`, project structure, and the Streamlit run command
- Screenshot or GitHub link showing the final commit and clean repository status

C. Future Improvements

- Restore API credits and compare live V1 and V2 outputs across multiple prospect websites.
- Add reliable source-level citations and excerpts for each finding.
- Add web search for current press releases, job postings, annual reports, and investor documents.
- Add structured competitor comparison and account-monitoring alerts.
- Add automated testing, caching, retries, rate limiting, authentication, and production deployment controls.
- Add meeting-brief and sales-deck generation from approved research sources and user-provided documents.